CSE499A – Section 15 – Group 03
Md. Rahad Hossain, Md. Yousuf, MD. Rokib Hasan Oli

CityPulse: Intelligent Parking & Traffic Control Platform

Weekly Progress Report – Update 6 (Sprint 6 in progress)

I. Sprint Overview

This week we made measurable progress toward an integrated CityPulse ecosystem by strengthening backend booking and inference coordination, improving frontend access control and session reliability, and advancing mobile real-time parking workflows. We focused on delivering a tighter end-to-end user experience across backend, web, and mobile clients, while documenting integration points and preserving reproducibility of the current prototype.

II. Backend & AI API Progress

Backend development centered on the bookings domain, slot occupancy inference, and production-ready upload and search support. The current implementation now connects booking lifecycle state with slot availability caching, while the inference scheduler updates live parking status from AI model predictions.

- Added a ‘bookings‘ module with secured CRUD endpoints, driver profile validation, auto-selection of available slots, and cancellation flows that preserve booking history.
- Integrated booking state into slot availability logic so ‘searchNearby‘ and parking slot responses now reflect both active bookings and AI inference occupancy.
- Implemented ‘InferenceSchedulerService‘ to run periodic cycles, fetch slot images via S3 presigned URLs, call the AI inference API, and update ‘parking_slots‘ occupancy with confidence scores.
- Enhanced ‘SpacesService‘ search to return nearby parking spaces with computed distances, slot-level status, stale-cache detection, and booking-aware availability counts.
- Added support for multiple photo uploads, RTSP camera registration during space creation, and secure presigned S3 URL generation for parking space photos.
- Extended authentication behavior by issuing longer-lived JWT access tokens via ‘JWT_EXPIRES_IN‘, refresh tokens with session tracking, and a public decorator to opt out protected routes where needed.
- Improved backend docs and release guidance in ‘applications/web/backend/README.md‘, including new environment configuration, AI inference endpoint parameters, and migration seed commands.

III. Frontend Progress

Frontend work reinforced the owner portal, authentication flows, and API integration contracts. The team improved routing correctness, session recovery behavior, and page-level access control for owners and administrators.

- Added portal profile pages, role-aware route guards, and a more consistent navigation experience across owner and admin workflows.
- Completed role-based app routing for protected pages and centralized the auth state handling to reduce duplicated logic.
- Fixed session persistence issues by clearing stale tokens on logout failure, ensuring users are redirected safely to the login flow.
- Improved responsive authentication layouts and removed an external font dependency to strengthen offline reliability.
- Connected parking lot components to backend APIs, enabling live status data for parking availability and slot details.
- Continued YOLOv8 model reproduction documentation with training notebooks, review notes, and frontend-facing reference materials for the PKLot experiments.
- Added more comprehensive documentation for screenshot assets and owner portal usage scenarios to support sprint review discussions.

IV. Mobile Progress

The mobile application advanced toward a polished parking management client with real API support and enhanced user flows. New features include live availability, history management, and booking-related UI improvements.

- Integrated nearby parking data and connected the dashboard to the real backend API, replacing placeholder data with live occupancy information.
- Added profile management features and refined the mobile brand with a new logo and updated app naming.
- Fixed booking and pricing workflows, including BDT price formatting issues and logout redirection to the landing page.
- Added the ability to delete parking history entries, improved search options, and stabilized the navbar experience across screens.
- Improved network resilience by adding fallback paths for API connectivity and securing Google API key handling in configuration.
- Updated the mobile booking flow to reflect real-time slot availability and reservation state from the backend service.

V. Documentation and Reproducibility

Documentation continued to be a cross-team priority, with coordinated updates across backend, frontend, and mobile project notes. The repository now includes more explicit integration references, API contract examples, and development artifacts for the current sprint.

- Updated the project README with deployment steps, feature branch notes, and backend service configuration guidance.
- Added explicit API contract documentation for booking and slot availability endpoints.
- Documented current mobile integration patterns for live parking status, search, and booking management.
- Preserved a demo video artifact and release notes for the sprint, ensuring the next review can highlight both technical progress and user-facing improvements.

- Continued to capture YOLOv8 PKLot reproduction notes and backend inference integration findings for reproducibility and future evaluation.

VI. Integration and Testing

We established a stronger integration focus by coordinating end-to-end validation scenarios across all three client surfaces. Key efforts included aligning API response formats, verifying authentication flow behavior across web and mobile, and preparing test cases for parking slot reservation synchronization.

- Aligned backend slot availability responses with frontend and mobile interpretation layers to eliminate mismatch bugs.
- Identified and documented cross-team test cases for booking creation, cancellation, and live status refresh.
- Prepared system-level validation scenarios for inference scheduling, API status updates, session recovery, and mobile booking flows.
- Began assembling a demo readiness checklist for the next sprint review, including live API walkthroughs and mobile experience snapshots.

VII. Plan for Next Week

- Explore: end-to-end testing strategies for distributed systems
- ☐ Build: integrate all components into a working end-to-end prototype
- ☐ Build: system testing, bug fixes, and demo preparation

References

[1] R. R. Prova *et al.*, "A Real-Time Parking Space Occupancy Detection Using Deep Learning Model," *IEEE IEMTRONICS*, 2022. DOI: 10.1109/IEMTRONICS55184.2022.9795771

[2] G. Amato *et al.*, "Deep learning for decentralized parking lot occupancy detection," *Expert Systems with Applications*, vol. 72, 2017.

[3] B. Yuldashev *et al.*, "Parking Lot Occupancy Detection with Improved MobileNetV3," *Sensors*, vol. 23, no. 17, 2023. DOI: 10.3390/s23177642